R ML/DL Libraries, Neural Networks, Transfer Learning & Multivariate Time Series

Data Analytics Using R and Python – 25B1BCA532

Aayush Sharma

Dept. of CSE & IT
Jaypee University of Information Technology, Waknaghat

Unit 4 – Advanced Topics

Lecture Roadmap

Part A: R Libraries for ML & DL

- The R DL ecosystem overview
- `keras` / `tensorflow` in R
- `torch` (libtorch via `luz`)
- `mlr3`, `caret`, `tidymodels` at a glance
- Choosing the right library

Part B: Basic Neural Networks

- Perceptron to MLP: the math
- Activation functions
- Backpropagation (step-by-step)
- Building & training in R (`keras`)
- Regularisation: Dropout, BatchNorm

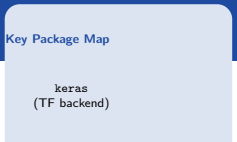
Part C: Transfer Learning

- What & why: intuition
- Fine-tuning vs. feature extraction
- Pretrained CNNs in R (`keras`)
- ARIMA basics and ARIMAX
- Hands-on: image classifier in R

Part D: Multivariate Time Series

- VAR models: theory & R
- Granger causality
- Cointegration & VECM
- Forecasting & IRF
- DL for MTS: LSTM in R

The R Deep Learning Ecosystem



Three main DL stacks in R:

1. `keras` (R) → calls Python `keras`/TensorFlow via `reticulate`
2. `torch` (R) → direct libtorch bindings, *no Python needed*
3. `mlr3torch` → integrates torch into the `mlr3` pipeline

Library	Paradigm	Best For
`keras` (R)	High-level	Quick prototyping, CNNs
`torch`	Low-level	Research, custom layers
`luz`	High-level torch	Training loops, callbacks
`mlr3`	Pipeline	Benchmarking, AutoML
`caret`	Classic ML	Tabular, cross-validation
`tidymodels`	Tidy ML	Production pipelines
`forecast`/`fable`	Time series	ARIMA, ETS, VAR

Installation Note

`keras` requires a Python environment:

```
install.packages("keras")
library(keras)
# First time:
install_keras()
# installs Python + TF
```

`torch` is standalone:

```
install.packages("torch")
library(torch)
# Downloads libtorch
# automatically
```

The keras Package in R – Core API

Sequential API (stacking layers linearly):

```r
library(keras)

model <- keras_model_sequential() |>
  layer_dense(units=128, activation="relu",
              input_shape=c(784)) |>
  layer_dropout(rate=0.3) |>
  layer_dense(units=64, activation="relu") |>
  layer_dense(units=10, activation="softmax")

model |> compile(
  optimizer = optimizer_adam(lr=0.001),
  loss      = "categorical_crossentropy",
  metrics   = c("accuracy")
)

summary(model)
```

Functional API (for multi-input/output):

```r
inputs  <- layer_input(shape=c(784))
x       <- inputs |> layer_dense(128, activation="
    relu")
outputs <- x |> layer_dense(10, activation="softmax")
model   <- keras_model(inputs, outputs)
```

Training & Evaluation

```r
history <- model |> fit(
  x_train, y_train,
  epochs     = 20,
  batch_size = 64,
  validation_split = 0.2,
  callbacks  = list(
    callback_early_stopping(
      monitor="val_loss",
      patience=3),
    callback_reduce_lr_on_plateau(
      factor=0.5, patience=2)
  )
)

# Visualise training curve
plot(history)

# Evaluate on test set
model |> evaluate(x_test, y_test)

# Predict
preds <- model |> predict(x_new)
```

The torch Package in R – Model Definition

A torch model is an nn_module:

```
library(torch)
library(luz)

MLP <- nn_module(
  "MLP",
  initialize = function(in_f, hidden, out_f) {
    self$fc1 <- nn_linear(in_f, hidden)
    self$fc2 <- nn_linear(hidden, out_f)
  },
  forward = function(x) {
    x <- self$fc1(x)
    x <- nnf_relu(x)
    self$fc2(x)
  }
)

model <- MLP(in_f=784, hidden=128, out_f=10)
```

Read the code

- `initialize()` creates layers and parameters.
- `forward()` defines how input flows through the model.
- `self$fc1` and `self$fc2` are trainable layers.
- ReLU adds non-linearity between the two dense layers.

Why use torch?

It is useful when you need custom layers, custom loss functions, easier debugging, or a deep learning stack that does not depend on Python.

Training `torch` Models with `luz`

`luz` gives `torch` a high-level training loop:

```
fitted <- model |>
  setup(
    loss = nn_cross_entropy_loss(),
    optimizer = optim_adam
  ) |>
  fit(train_dl,
      epochs = 20,
      valid_data = val_dl)

preds <- predict(fitted, test_dl)
```

Tensor operations and device selection:

```
x <- torch_randn(3, 4)
z <- torch_mm(x, torch_ones(4, 2))
device <- if(cuda_is_available()) "cuda" else "
    cpu"
x <- x$to(device=device)
```

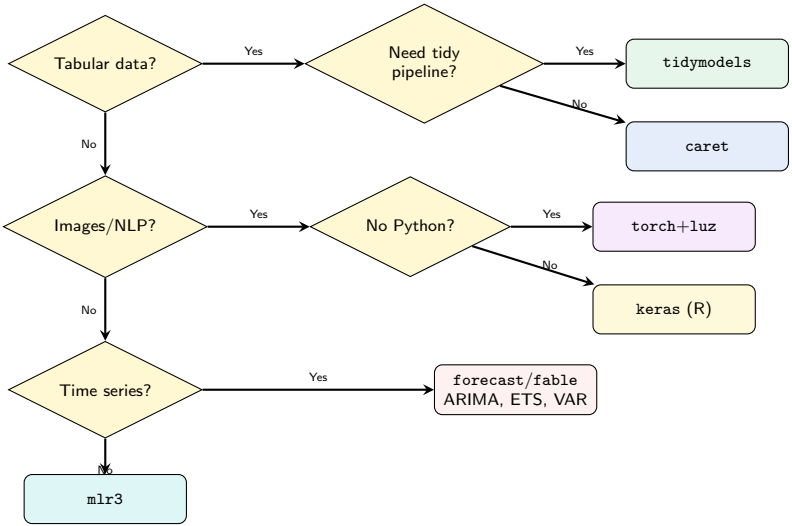
`torch` vs `keras`

Aspect	keras	torch
Python needed	Yes	No
Style	High-level	Flexible
Graph	More fixed	Dynamic
Custom layers	Medium	Strong
Debugging	Harder	Easier

Exam point

`torch` is lower-level than `keras`. It gives more control, but the user must understand layers, tensors, loss functions, and training loops more explicitly.

Choosing the Right R Library – Decision Flow



Choosing the Right R Library – Practical Guide

Tabular ML

`tidymodels`: best for clean production pipelines, recipes, tuning, and tidy workflows.

`caret`: classic package for quick models, cross-validation, and many algorithms.

Deep Learning

`keras`: easiest for CNNs, image models, and quick prototypes; uses TensorFlow backend.

`torch+luz`: better for custom layers and no-Python workflows.

Time Series

`forecast/fable`: ARIMA, ETS, decomposition, and forecasting workflows.

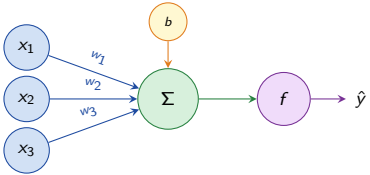
`vars/urca`: VAR, VECM, Granger causality, cointegration.

Selection rule

Start with the simplest package that matches the data type. Use deep learning only when data volume and problem complexity justify it.

From Perceptron to Multi-Layer Perceptron

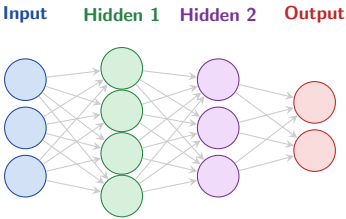
Single Perceptron



$$\text{net} = \sum_i w_i x_i + b$$

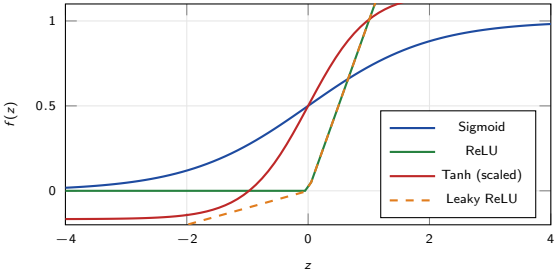
$$\hat{y} = f(\text{net})$$

Multi-Layer Perceptron (MLP)



Universal Approximation Theorem: An MLP with a single hidden layer and enough neurons can approximate *any* continuous function on a compact domain.

Activation Functions – The Non-Linearity Engine



Activation	Use case
Sigmoid $\frac{1}{1+e^{-z}}$	Binary output (last layer)
Softmax $\frac{e^{z_i}}{\sum e^{z_j}}$	Multi-class output
Tanh $\frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$	Hidden layers (zero-centered)
ReLU $\max(0, z)$	Hidden layers (default)
Leaky ReLU	Prevent dying neurons
GELU	Transformers, BERT

Dying ReLU Problem

If a neuron's input is always negative, ReLU outputs 0 and gradient is 0 – the neuron *never* learns. Fix: Leaky ReLU or ELU.

Backpropagation – The Core Algorithm

Forward Pass: compute predictions layer by layer.
Loss: $\mathcal{L} = \frac{1}{N}\sum_i (y_i - \hat{y}_i)^2$ (MSE) or cross-entropy.
Backward Pass: apply chain rule to propagate $\frac{\partial \mathcal{L}}{\partial w}$ backwards:

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$

Weight update: $w \leftarrow w - \eta \cdot \frac{\partial \mathcal{L}}{\partial w}$

Optimisers in R/keras

- `optimizer_sgd(lr)` – basic gradient descent
- `optimizer_adam(lr)` – adaptive, default choice
- `optimizer_rmsprop(lr)` – good for RNNs
- `optimizer_adagrad(lr)` – sparse features

Numerical Example: 1 input x=0.5, 1 hidden (ReLU), 1 output (linear). w_1=0.4, w_2=0.6, biases= 0, target y=1, η=0.1.
Forward:

$$z_1 = 0.4 \times 0.5 = 0.2, \quad a_1 = \text{ReLU}(0.2) = 0.2$$

$$z_2 = 0.6 \times 0.2 = 0.12 = \hat{y}, \quad \mathcal{L} = (1 - 0.12)^2 = 0.7744$$

Backward (chain rule):

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = -1.76, \quad \frac{\partial \mathcal{L}}{\partial w_2} = -1.76 \times 0.2 = -0.352$$

$$\frac{\partial \mathcal{L}}{\partial a_1} = -1.056, \quad \frac{\partial \mathcal{L}}{\partial w_1} = -1.056 \times 0.5 = -0.528$$

Update:

$$w_2 \leftarrow 0.6 - 0.1 \times (-0.352) = \mathbf{0.6352}$$

$$w_1 \leftarrow 0.4 - 0.1 \times (-0.528) = \mathbf{0.4528}$$

After one update, $\hat{y}$ becomes $0.6352 \times (0.4528 \times 0.5) = 0.144$, closer to target $y = 1$. Repeat for thousands of steps.

Building an MLP in R with keras – MNIST Example

```r
library(keras)
mnist   <- dataset_mnist()

# Preprocess: flatten + normalise
x_train <- array_reshape(mnist$train$x,
             c(60000, 784)) / 255
x_test  <- array_reshape(mnist$test$x,
             c(10000, 784)) / 255
y_train <- to_categorical(mnist$train$y, 10)
y_test  <- to_categorical(mnist$test$y,  10)

# Build model
model <- keras_model_sequential() |>
  layer_dense(512, activation="relu",
              input_shape=c(784)) |>
  layer_batch_normalization() |>
  layer_dropout(0.4) |>
  layer_dense(256, activation="relu") |>
  layer_dropout(0.3) |>
  layer_dense(10, activation="softmax")

model |> compile(
  optimizer = optimizer_adam(lr=5e-4),
  loss      = "categorical_crossentropy",
  metrics   = "accuracy"
)
```

```r
# Train with early stopping
history <- model |> fit(
  x_train, y_train,
  epochs=15, batch_size=128,
  validation_split=0.1,
  callbacks=list(
    callback_early_stopping(
      patience=3,
      restore_best_weights=TRUE))
)
plot(history)  # training curve

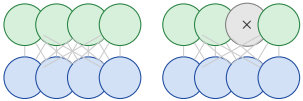
# Evaluate
results <- model |>
  evaluate(x_test, y_test)
cat("Test Acc:", results["accuracy"])
# Typically ~98.5%

# Confusion matrix
preds <- model |> predict(x_test) |>
         apply(1, which.max) - 1
library(caret)
confusionMatrix(factor(preds),
                factor(mnist$test$y))
```

Regularisation: Dropout & Batch Normalisation

Dropout

Without Dropout **With Dropout (p=0.5)**



Dropout randomly *zeroes p* fraction of neurons during training, preventing co-adaptation.

```
# Applied AFTER activation
layer_dense(256, activation="relu") |>
layer_dropout(rate=0.4)
```

Batch Normalisation

Normalises within each mini-batch, then applies learned γ, β:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

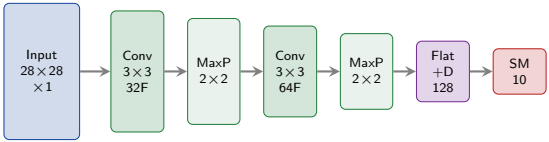
Benefits: faster training, higher LR, less internal covariate shift.

```
# BN before activation:
layer_dense(256) |>
layer_batch_normalization() |>
layer_activation("relu")

# Or after (also common):
layer_dense(256, activation="relu") |>
layer_batch_normalization()
```

Rule of thumb: Dropout on FC layers; BatchNorm on CNNs/RNNs. Both reduce overfitting and speed up convergence.

Convolutional Neural Networks (CNNs) in R




```
model <- keras_model_sequential() |>
  layer_conv_2d(32, c(3,3),
    activation="relu",
    input_shape=c(28,28,1)) |>
  layer_max_pooling_2d(c(2,2)) |>
  layer_conv_2d(64, c(3,3),
    activation="relu") |>
  layer_max_pooling_2d(c(2,2)) |>
  layer_flatten() |>
  layer_dense(128, activation="relu") |>
  layer_dropout(0.5) |>
  layer_dense(10, activation="softmax")
```

CNN Key Concepts

- **Filter/Kernel**: Small weight matrix slides over input computing dot products. A 3×3 filter has $9 + 1 = 10$ parameters.
- **Feature map**: Output of one filter applied across the image.
- **Max Pooling**: Takes max in each 2×2 window, halving spatial dimensions.
- **Channels**: After conv, #channels = #filters.
- **Parameter sharing**: Same filter weights used at every spatial location – huge parameter reduction.

Parameter count example:
Conv layer: 32 filters of $3 \times 3 \times 1$ + bias
$= 32 \times (9 + 1) = 320$ parameters
Dense layer: $1600 \rightarrow 128$
$= 1600 \times 128 + 128 = 205{,}056$ parameters
$\Rightarrow$ CNNs are efficient!

Puzzle 1 – Debug the Neural Network

Puzzle

The following code has **4 bugs**. Find and fix them.

```
library(keras)
mnist   <- dataset_mnist()
x_train <- mnist$train$x / 255     # (A)
y_train <- mnist$train$y            # (B)

model <- keras_model_sequential() |>
  layer_dense(128, activation="relu",
    input_shape=c(28, 28)) |>      # (C)
  layer_dense(10, activation="sigmoid") # (D)

model |> compile(
  optimizer="adam",
  loss="categorical_crossentropy",
  metrics="accuracy")
model |> fit(x_train, y_train, epochs=5)
```

Puzzle 1 – Debug the Neural Network

Puzzle

The following code has **4 bugs**. Find and fix them.

```
library(keras)
mnist   <- dataset_mnist()
x_train <- mnist$train$x / 255     # (A)
y_train <- mnist$train$y            # (B)

model <- keras_model_sequential() |>
  layer_dense(128, activation="relu",
    input_shape=c(28, 28)) |>       # (C)
  layer_dense(10, activation="sigmoid") # (D)

model |> compile(
  optimizer="adam",
  loss="categorical_crossentropy",
  metrics="accuracy")
model |> fit(x_train, y_train, epochs=5)
```

Solutions

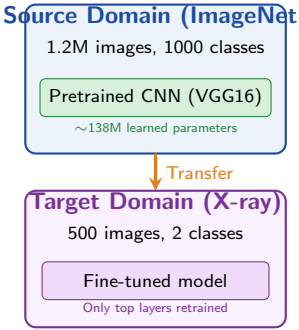
(A) `x_train` is 3D $(60000, 28, 28)$ but Dense needs 2D. Fix: `array_reshape(x_train, c(60000, 784))`

(B) Labels are integers; loss expects one-hot. Fix: `to_categorical(y_train, 10)`

(C) After reshape to 784, use `input_shape=c(784)`, not `c(28,28)`

(D) `sigmoid` treats classes independently. Fix: `activation="softmax"` for mutual exclusivity

What is Transfer Learning?



Why Transfer Learning?

- Training deep CNNs from scratch requires millions of labelled examples and weeks of GPU time
- Lower layers learn *universal features*: edges, textures, curves
- Higher layers learn task-specific patterns
- Transfer: keep low-level features, retrain task-specific layers
- Often achieves state-of-the-art with **hundreds** of examples

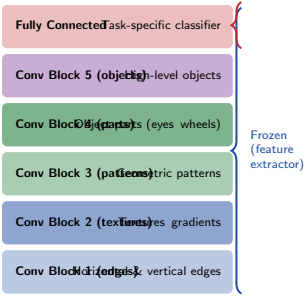
Two Strategies

Feature Extraction: Freeze *all* pretrained layers, add new classifier head, only train head.

Fine-tuning: Unfreeze *some* top pretrained layers, retrain with a very small learning rate.

Transfer Learning – Inside the Network

What each layer learns (VGG16 on ImageNet):



Feature Extraction vs Fine-tuning

	Feat. Extract.	Fine-tune
Pretrained layers	All frozen	Top unfrozen
Learning rate	Normal	Very small
Dataset size	$\geq$100	$\geq$500
Risk of overfit	Low	Medium
Performance	Good	Better
Training speed	Fast	Slower

When Does Transfer Learning Work?

- **Similar domains:** Natural images $\rightarrow$ medical scans (works well; both are 2D spatial)
- **Different domains:** Satellite images $\rightarrow$ microscopy (still useful; low-level features transfer)
- **Small dataset:** Even 200 examples can fine-tune effectively
- **Dissimilar task:** Only freeze early layers, fine-tune more

Transfer Learning in R with keras – VGG16

Steps 1 & 2: Load base + add head

```
library(keras)
# Load VGG16, no top classifier
base_model <- application_vgg16(
  weights="imagenet", include_top=FALSE,
  input_shape=c(224, 224, 3))
freeze_weights(base_model)  # freeze all

# Add custom classifier head
inputs  <- base_model$input
x       <- base_model$output |>
  layer_global_average_pooling_2d() |>
  layer_dense(256, activation="relu") |>
  layer_dropout(0.5)
outputs <- x |>
  layer_dense(1, activation="sigmoid")
model <- keras_model(inputs, outputs)
```

Step 3: Train only the head

```
model |> compile(
  optimizer=optimizer_adam(lr=1e-4),
  loss="binary_crossentropy",
  metrics="accuracy")
model |> fit(train_gen, epochs=10,
             validation_data=val_gen)
```

Step 4: Fine-tune top conv layers

```
# Unfreeze last conv block
unfreeze_weights(base_model,
                 from="block5_conv1")
# MUCH smaller LR to avoid destroying
# pretrained weights
model |> compile(
  optimizer=optimizer_adam(lr=1e-5),
  loss="binary_crossentropy",
  metrics="accuracy")
model |> fit(train_gen, epochs=5,
             validation_data=val_gen)
```

Why lr=10^{-5}? Pretrained weights encode millions of examples — a large LR overwrites them in a few steps. Always freeze base → train head → unfreeze top → fine-tune with tiny LR.

Data Augmentation & Pretrained Model Selection

```r
library(keras)
# Training: augment + rescale
train_gen <- image_data_generator(
  rescale=1/255, rotation_range=20,
  width_shift_range=0.2,
  height_shift_range=0.2,
  zoom_range=0.2, horizontal_flip=TRUE,
  fill_mode="nearest")
# Validation: ONLY rescale
val_gen <- image_data_generator(rescale=1/255)

train_flow <- flow_images_from_directory(
  "data/train", generator=train_gen,
  target_size=c(224,224), batch_size=32,
  class_mode="binary")
```

Never augment the validation set. Augmenting val gives artificially lower and noisy accuracy – you lose the ability to diagnose overfitting.

Pretrained Models in `keras`

Function	Best for
`application_vgg16()`	Baseline
`application_resnet50()`	General use
`application_mobilenet()`	Edge/mobile
`application_inception_v3()`	Multi-scale
`application_densenet121()`	Medical img
`application_efficientnet_b0()`	Best ratio

Choosing a Base Model

- **EfficientNetB0**: best accuracy-per-parameter trade-off overall
- **MobileNetV2**: when inference speed matters (mobile/IoT)
- **ResNet50**: reliable, well-understood baseline
- **DenseNet121**: strong for medical imaging tasks
- All use `weights="imagenet"` for pretrained weights

ARIMA: What Problem Does It Solve?

Core idea

ARIMA is used for **univariate time-series forecasting**: predicting the future value of one variable from its own past.

Example questions:

- What will next month's sales be?
- What will electricity demand be tomorrow?
- What will the stock index level be next week?

ARIMA is suitable when observations are ordered in time and nearby observations are related.

The modelling story

1. Remove trend/non-stationarity by differencing.
2. Model the remaining dependence using past values.
3. Correct using past forecast errors.
4. Forecast future values and attach uncertainty bands.

Important

ARIMA does not use external variables. If sales depend on ad spend, price, rainfall, or holidays, use ARIMAX/SARIMAX or another regression-with-time-series-error model.

ARIMA(p, d, q): Meaning of Each Part

General ARIMA idea

Let y_t be the original series and let z_t be the differenced series:

$$z_t = (1 - B)^d y_t$$

The ARMA model is then fitted to z_t:

$$z_t = c + \phi_1 z_{t-1} + \cdots + \phi_p z_{t-p} + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q}$$

B is the backshift operator: $By_t = y_{t-1}$.

Meaning of p, d, q

p	AR order: number of lagged values of the differenced series
d	Differencing order: number of times trend is removed
q	MA order: number of lagged forecast errors used

How to read names

ARIMA(1,1,1) means: difference once, then use one AR lag and one MA error lag.

ARIMA(0,1,0) means: a random walk after first differencing.

Step 1 in ARIMA: Differencing by Hand

Why difference?

ARIMA assumes the modelled series is roughly **stationary**: its mean, variance, and autocorrelation pattern do not keep changing over time. A trend violates this, so we difference:

$$\Delta y_t = y_t - y_{t-1}$$

Do not over-difference

Use $d = 1$ when one difference removes the trend. Use $d = 2$ only if the first-differenced series still has a clear trend.

Numerical differencing

t	y_t	Δy_t	Meaning
1	100	–	first value
2	104	4	$104 - 100$
3	107	3	$107 - 104$
4	113	6	$113 - 107$
5	120	7	$120 - 113$
6	128	8	$128 - 120$

The original series rises steadily; the differences are much more stable. This suggests starting with $d = 1$.

Choosing p and q: ACF/PACF Rules

ACF

The autocorrelation function measures correlation between z_t and z_{t-k} for different lags k.

If ACF has a sharp cut-off after lag q, it suggests an MA(q) part.

PACF

The partial autocorrelation function measures the direct relationship at lag k after removing smaller lags.

If PACF cuts off after lag p, it suggests an AR(p) part.

Quick identification table

Pattern	ACF	PACF
AR(p)	Tails off	Cuts after p
MA(q)	Cuts after q	Tails off
ARMA	Tails off	Tails off
White noise	Near zero	Near zero

In practice, compare candidate models using AIC/BIC and then check residuals. ACF/PACF gives a starting point, not the final answer.

Numerical Example: Forecast with ARIMA(1,1,0)

Given data

Monthly sales:

$$y_1 = 100,\ y_2 = 104,\ y_3 = 107,\ y_4 = 113,\ y_5 = 120,\ y_6 = 128$$

First differences:

$$z_2 = 4,\ z_3 = 3,\ z_4 = 6,\ z_5 = 7,\ z_6 = 8$$

Suppose the fitted model for differences is:

$$z_t = 1.5 + 0.7 z_{t-1} + \varepsilon_t$$

One-step forecast

Forecast the next difference:

$$\hat{z}_7 = 1.5 + 0.7 z_6 = 1.5 + 0.7(8) = 7.1$$

Convert back to original sales:

$$\hat{y}_7 = y_6 + \hat{z}_7 = 128 + 7.1 = 135.1$$

Because $d = 1$, the model forecasts the next *change*; we add the predicted change to the latest actual value.

Exam-Style Numerical Problem: Full Solution

Puzzle

Sales for six months are $100, 104, 107, 113, 120, 128$. A fitted ARIMA(1,1,0) model has

$$\Delta y_t = 1.5 + 0.7\Delta y_{t-1} + \varepsilon_t.$$

Find forecasts for months 7 and 8.

Month 7

Last observed difference:

$$\Delta y_6 = 128 - 120 = 8$$

$$\widehat{\Delta y_7} = 1.5 + 0.7(8) = 7.1$$

$$\hat{y}_7 = 128 + 7.1 = 135.1$$

Month 8

Use the month-7 forecasted difference as the lag:

$$\widehat{\Delta y_8} = 1.5 + 0.7(7.1) = 6.47$$

$$\hat{y}_8 = \hat{y}_7 + 6.47 = 135.1 + 6.47 = 141.57$$

Fitting ARIMA in R – Complete Workflow

```r
library(forecast)

sales <- ts(c(100,104,107,113,120,128,
              135,139,145,151,156,160),
            frequency = 12,
            start = c(2024, 1))

# Inspect trend/seasonality first
plot(sales)

# Let R search p, d, q
fit <- auto.arima(sales,
                  seasonal = FALSE,
                  stepwise = FALSE)
summary(fit)

# Residuals must be white noise
checkresiduals(fit)

# Forecast next 6 months
fc <- forecast(fit, h = 6)
plot(fc)
```

What to write in an answer

- Mention selected order, e.g. ARIMA(1,1,0).
- Explain why differencing was used.
- Report AIC/BIC for comparison.
- Check residual ACF and Ljung-Box test.
- Present forecast and prediction interval.

Common mistake

If residuals still show autocorrelation, the model has missed time-series structure. Do not trust the forecast until diagnostics are acceptable.

ARIMA vs ARIMAX: When Do We Add External Inputs?

ARIMA

Use ARIMA when the target is mainly explained by its own past.

$$\text{sales}_t \leftarrow \text{past sales} + \text{past errors}$$

Example: forecasting store sales only from previous store sales.

ARIMAX

Use ARIMAX when another variable helps explain the target.

$$\text{sales}_t \leftarrow \text{past sales} + \text{ad spend}_t + \text{errors}$$

Example: forecasting sales using past sales and planned advertising spend.

Bridge to transfer-function models

ARIMAX is not just ordinary regression. The external input can affect the target immediately and with lagged effects, so the response is modelled over time.

Transfer Function Models (ARIMAX)

Transfer Function Model = ARIMA + external input x_t

ARIMAX Equation

$$y_t = \underbrace{\sum_{k=0}^{r} \nu_k x_{t-k}}_{\text{transfer function}} + \underbrace{\frac{\theta(B)}{\phi(B)}\varepsilon_t}_{\text{ARIMA noise}}$$

where B is the backshift operator: $B^k y_t = y_{t-k}$.

Why not simple regression + ARIMA?

A plain regression ignores *lagged* effects of x_t on y_t. The transfer function $\nu(B) = \nu_0 + \nu_1 B + \ldots$ captures how y responds to x over multiple periods.

Fitting steps: (1) Pre-whiten y_t, x_t with common ARIMA filter; (2) CCF to identify lags; (3) Estimate ν_k; (4) ARIMA on residuals; (5) Check residuals for white noise.

R Implementation

```r
library(forecast)

# Fit ARIMAX (ARIMA + external regressor)
fit <- auto.arima(
  y_sales,
  xreg = x_adspend
)
summary(fit)
# AR(1), MA(1) + xreg coefficient

# Forecast with future x values
fc <- forecast(fit,
  xreg = c(50, 55, 60),  # planned ad
  h    = 3)
plot(fc)

# Residual diagnostics
checkresiduals(fit)
# Ljung-Box p > 0.05 = white noise OK
```

Puzzle 2 – Transfer Learning Strategy

Puzzle

A student has 200 chest X-ray images (100 normal, 100 pneumonia). They plan to train a **full ResNet50 from scratch** for 50 epochs.

(a) Identify **two** specific problems with this approach.
(b) Write the corrected R/keras code using transfer learning.
(c) The student sees 99% training accuracy but 58% validation accuracy after epoch 3. What is happening and what is the **single most important** fix?

Puzzle 2 – Solution

Solution (a) & (c)

(a) Problems:

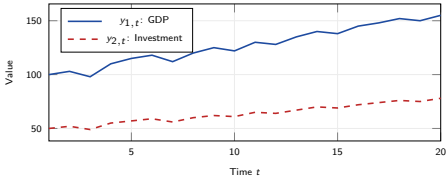
1. 200 images is far too few for ResNet50 (25M params) from scratch – extreme overfitting guaranteed
2. Training ResNet50 from scratch requires millions of GPU-hours; impractical

(c) This is **severe overfitting**. The model memorised 200 training examples. Most important fix: use a pretrained model with frozen base layers (feature extraction). Also: data augmentation.

Solution (b)

```
base <- application_resnet50(
  weights="imagenet",
  include_top=FALSE,
  input_shape=c(224,224,3))
freeze_weights(base)
inputs  <- base$input
x       <- base$output |>
  layer_global_average_pooling_2d() |>
  layer_dense(64,"relu") |>
  layer_dropout(0.5)
outputs <- x |>
  layer_dense(1,"sigmoid")
model <- keras_model(inputs, outputs)
model |> compile("adam",
  "binary_crossentropy","accuracy")
```

Multivariate Time Series – Motivation & Concepts



Univariate TS: One variable at a time – ARIMA, ETS.

Multivariate TS: Multiple variables interact and *cause each other*. GDP affects investment; investment feeds back into GDP.

Key question: Can past values of y_2 improve forecasts of y_1?

Key MTS Models

Model	Purpose
VAR	All vars mutually cause each other; stationary series
VECM	VAR with cointegration (series are co-integrated)
ARIMAX	ARIMA + external predictors
LSTM	Deep learning for non-linear MTS
VARMA	VAR + MA terms

Stationarity Reminder

MTS methods assume **stationarity** (constant mean, variance, covariance). Test each series:

- ADF test: `adf.test(y)`
- KPSS test: `kpss.test(y)`
- If non-stationary: difference with `diff(y)`

Vector Autoregression (VAR) – Theory

VAR(p) model for K variables at lag p:

$$\mathbf{y}_t = \mathbf{c} + \mathbf{A}_1 \mathbf{y}_{t-1} + \mathbf{A}_2 \mathbf{y}_{t-2} + \cdots + \mathbf{A}_p \mathbf{y}_{t-p} + \boldsymbol{\varepsilon}_t$$

where $\mathbf{y}_t = (y_{1,t}, y_{2,t})^\top$, $\mathbf{A}_i$ are $K \times K$ coefficient matrices, $\boldsymbol{\varepsilon}_t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma})$.

For $K = 2$, $p = 1$ expanded:

$$\begin{pmatrix} y_{1,t} \\ y_{2,t} \end{pmatrix} = \begin{pmatrix} c_1 \\ c_2 \end{pmatrix} + \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \begin{pmatrix} y_{1,t-1} \\ y_{2,t-1} \end{pmatrix} + \begin{pmatrix} \varepsilon_{1,t} \\ \varepsilon_{2,t} \end{pmatrix}$$

- $a_{12} \neq 0$ means lagged y_2 predicts y_1 (*Granger causality*)
- $a_{21} \neq 0$ means lagged y_1 predicts y_2
- All equations estimated by OLS

Selecting Lag Order p

Use information criteria:

$$\text{AIC} = -2 \ln \hat{L} + 2k \quad \text{(penalise complexity)}$$

$$\text{BIC} = -2 \ln \hat{L} + k \ln T \quad \text{(stricter)}$$

Choose p that minimises AIC or BIC.

Granger Causality

y_2 **Granger-causes** y_1 if past values of y_2 significantly improve the prediction of y_1 beyond what y_1's own past provides.

Test: F-test on whether coefficients $a_{12}, a_{12}^{(2)}, \ldots$ are jointly zero.

Caution: Granger causality $\neq$ true causality! It is predictive causality.

VAR in R – Complete Worked Example

Data: Canadian macroeconomic data (GDP, investment)

```r
library(vars)
library(tseries)

# 1. Load data (built-in in 'vars' package)
data(Canada)
# Canada: e (employment), prod, rw, U (unemp)
y <- Canada[, c("prod", "e")]

# 2. Check stationarity
adf.test(y[,"prod"])  # p < 0.05 => stationary
adf.test(y[,"e"])

# 3. Select lag order
lag_sel <- VARselect(y, lag.max=8,
                     type="const")
lag_sel$selection
# AIC selects p=3 (example)

# 4. Fit VAR(3)
var_fit <- VAR(y, p=3, type="const")
summary(var_fit)
```

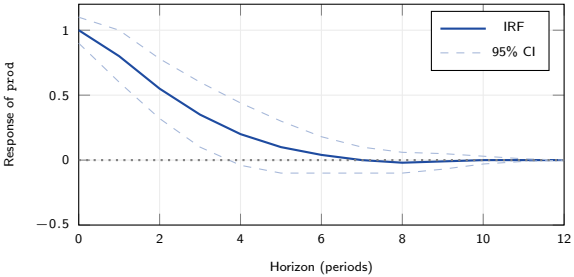
```r
# 5. Granger causality tests
causality(var_fit, cause="e")
# Tests: does employment Granger-cause prod?
# H0: No Granger causality
# Reject H0 if p < 0.05

# 6. Impulse Response Function
# How does prod respond to shock in e?
irf_result <- irf(var_fit,
                  impulse="e",
                  response="prod",
                  n.ahead=12,
                  boot=TRUE)
plot(irf_result)

# 7. Forecast (h=8 steps ahead)
fc <- predict(var_fit, n.ahead=8,
              ci=0.95)
plot(fc)

# 8. Check VAR stability
roots(var_fit)
# All moduli < 1 => stable
```

Impulse Response Function (IRF) – Interpretation



Reading an IRF

- X-axis: number of periods after the shock
- Y-axis: response magnitude (in units of the response variable)
- Solid line: estimated response
- Dashed lines: bootstrap 95% CI
- **If CI crosses zero**: effect not statistically significant at that horizon
- **If CI stays above zero**: positive, significant effect

Interpretation above: A one-SD shock to employment (`e`) causes productivity (`prod`) to rise for about 5 periods before dying out. The effect is significant at horizons 0–3 (CI above zero) but not at 5+ periods.

Cointegration & Vector Error Correction Model (VECM)

What is Cointegration?

Two non-stationary series $y_{1,t}$ and $y_{2,t}$ are **cointegrated** if there exists β such that:

$$e_t = y_{1,t} - \beta y_{2,t} \quad \text{is stationary (I(0))}$$

Meaning: they are tied together in the long run even though each is non-stationary individually.

Example: Stock price of Reliance and its earnings per share both have unit roots, but the price-to-earnings ratio is stationary.

Interpretation

If the series move away from their long-run relationship, the error-correction term pulls them back over future periods.

When to use it

- Each series is non-stationary, usually I(1)
- A stationary linear combination exists
- You need both short-run changes and long-run equilibrium

VECM is a VAR that imposes the cointegration restriction:

$$\Delta \mathbf{y}_t = \underbrace{\boldsymbol{\alpha}\boldsymbol{\beta}^\top \mathbf{y}_{t-1}}_{\text{error correction}} + \sum_{i=1}^{p-1} \boldsymbol{\Gamma}_i \Delta \mathbf{y}_{t-i} + \boldsymbol{\varepsilon}_t$$

- $\boldsymbol{\beta}^\top \mathbf{y}_{t-1}$: long-run equilibrium relationship ("error")
- $\boldsymbol{\alpha}$: speed of adjustment back to equilibrium

VECM in R – Johansen Test and Forecast

VECM in R

```
library(urca)
library(vars)

# 1. Test for cointegration
# Johansen trace test
jo <- ca.jo(y,
            type  = "trace",
            ecdet = "const",
            K     = 2)
summary(jo)
# r=0: no cointegration
# r<=1: at most 1 cointegrating vector

# 2. If cointegrated, fit VECM
vecm <- cajorls(jo, r=1)
# r=1 cointegrating relationship

# 3. Convert VECM -> VAR
var_from_vecm <- vec2var(jo, r=1)

# 4. Forecast from VAR form
fc <- predict(var_from_vecm, n.ahead=8)
plot(fc)
```

Decision rule:
If series are I(1) **and** cointegrated ⇒ use VECM.
If series are I(1) **but not** cointegrated ⇒ difference first, then VAR.
If series are I(0) ⇒ use VAR in levels.

LSTM for Multivariate Time Series in R

Why LSTM?

VAR/VECM are **linear** – they miss non-linear dynamics. LSTM maintains a **memory cell state** C_t gated by three learned gates:

Gate	Role
Forget $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$	Erase old memory
Input $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$	Write new info
Output $o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$	Read from memory
$C_t = f_t \odot C_{t-1} + i_t \odot \tanh(W_C[\ldots])$	
$h_t = o_t \odot \tanh(C_t)$	

LSTM input shape: `[samples, timesteps, features]` – a **3D array**, not a matrix. This is the most common error.

```r
library(keras)
n_steps <- 10; n_feat <- 3

# Sliding window: 3D array
X <- array(NA, c(nrow(data)-n_steps,
                 n_steps, n_feat))
Y <- matrix(NA, nrow(data)-n_steps, n_feat)
for(i in seq_len(nrow(data)-n_steps)){
  X[i,,] <- as.matrix(
              data[i:(i+n_steps-1),])
  Y[i,]  <- as.numeric(data[i+n_steps,])
}

# Stacked LSTM model
model <- keras_model_sequential() |>
  layer_lstm(64, return_sequences=TRUE,
    input_shape=c(n_steps, n_feat)) |>
  layer_dropout(0.2) |>
  layer_lstm(32) |>
  layer_dense(n_feat)

model |> compile(optimizer="adam",
                 loss="mse")
model |> fit(X, Y, epochs=50,
  batch_size=16, validation_split=0.2)
```

VAR vs LSTM – When to Use Which?

Aspect	VAR	LSTM
Linearity	Linear	Non-linear
Min sample size	∼100	∼1000+
Interpretability	High	Low
Granger causality	Yes	No
Compute cost	Low	High (GPU)
Short series	Excels	Overfits
Long series	Misses patterns	Excels
Structural breaks	Struggles	Robust

Rule of thumb: Benchmark VAR first. For macro/financial data with <500 obs, VAR often beats LSTM. LSTM wins on high-frequency data (daily/hourly, thousands of obs).

```r
library(vars); library(Metrics)

# 80/20 split
n_tr <- floor(0.8 * nrow(y_mat))
y_tr <- y_mat[1:n_tr, ]
y_te <- y_mat[(n_tr+1):nrow(y_mat), ]

# -- VAR benchmark --
var_m  <- VAR(y_tr, p=3, type="const")
fc_var <- predict(var_m,
            n.ahead=nrow(y_te))$fcst
rmse_var <- rmse(y_te[,"GDP"],
                 fc_var$GDP[,"fcst"])

# -- LSTM benchmark --
# (build + train as previous slide)
lstm_fc   <- predict(lstm_model, X_te)
rmse_lstm <- rmse(y_te[,"GDP"],
                  lstm_fc[,1])

cat("VAR RMSE: ", rmse_var,
    "\nLSTM RMSE:", rmse_lstm)
```

Puzzle 3 – VAR Model Diagnosis

Puzzle

VAR(2) on `price` (stock) & `vol` (volume), 260 obs:

```
# ADF: price p=0.41 (unit root), vol p=0.03 (stationary)
var_fit <- VAR(cbind(price, vol), p=2, type="const")
roots(var_fit)    # [1] 0.98  0.97  0.45  0.23
causality(var_fit, cause="vol")$Granger  # p=0.002
```

(a) Is VAR specification correct?
(b) What do roots near 1 indicate?
(c) Does p=0.002 mean vol **causes** price changes?

Puzzle 3 – VAR Model Diagnosis

Puzzle

VAR(2) on `price` (stock) & `vol` (volume), 260 obs:

```
# ADF: price p=0.41 (unit root), vol p=0.03 (stationary)
var_fit <- VAR(cbind(price, vol), p=2, type="const")
roots(var_fit)    # [1] 0.98  0.97  0.45  0.23
causality(var_fit, cause="vol")$Granger  # p=0.002
```

(a) Is VAR specification correct?
(b) What do roots near 1 indicate?
(c) Does p=0.002 mean vol **causes** price changes?

Solutions

(a) No. `price` is I(1), `vol` is I(0) – different integration orders. Must first difference `price` (or use VECM if cointegrated).

(b) Roots $\geq$0.97 mean VAR is near non-stationary – explosive forecasts, inflated IRFs. This confirms misspecification.

(c) No. Granger = *predictive* causality only. Both series may be driven by a common factor (e.g. sentiment). Also model is misspecified, so the p-value is unreliable.

Seasonal Decomposition & Pre-processing for MTS

```r
library(forecast); library(tseries)

# 1. STL decomposition
ts_data    <- ts(price_data, frequency=52)
stl_decomp <- stl(ts_data, s.window="periodic")
sa         <- seasadj(stl_decomp) # remove season

# 2. Difference to achieve stationarity
price_diff <- diff(sa)
adf.test(price_diff)  # should be p < 0.05

# 3. Scale (TRAINING data only -- avoid leakage)
tr_mean <- colMeans(y_tr)
tr_sd   <- apply(y_tr, 2, sd)
y_tr_sc <- scale(y_tr, center=tr_mean,
                       scale=tr_sd)
y_te_sc <- scale(y_te, center=tr_mean,
                       scale=tr_sd)

# Invert after prediction:
y_hat_orig <- y_hat_scaled * tr_sd + tr_mean
```

Data Leakage in Scaling

WRONG: `scale(y_mat)` – uses all rows including test

CORRECT: Fit scaler on train only, apply same params to test. The model must never see test statistics during training.

MTS Pre-processing Checklist

1. ADF/KPSS test each variable
2. Check ACF for seasonality
3. STL decompose if seasonal
4. Difference non-stationary series
5. Johansen test if using VAR on I(1)
6. Lag order: `VARselect()` AIC/BIC
7. Post-fit: residual ACF should be white noise

Puzzle 4 – Multivariate Time Series End-to-End

Puzzle

Monthly 2015–2024: `inf` (CPI), `repo` (rate), `iip` (production). All I(1).

(a) Write R code: Johansen test, VAR vs VECM decision, fit model, forecast 6 months.

(b) `repo` Granger-causes `inf` (p=0.01). Does this prove raising repo lowers inflation?

(c) IRF of `iip` to `inf` shock: CI is very wide, crosses zero everywhere. Interpret & fix.

```r
library(vars); library(urca)
y  <- cbind(inf, repo, iip)
jo <- ca.jo(y, type="trace",
            ecdet="const", K=3)
summary(jo)
# r>0: VECM; r=0: VAR on diff(y)
if(r > 0){
  v <- vec2var(jo, r=r)
} else {
  v <- VAR(apply(y,2,diff),
           p=2, type="const")
}
plot(predict(v, n.ahead=6))
```

Puzzle 4 – Multivariate Time Series End-to-End

Puzzle

Monthly 2015–2024: `inf` (CPI), `repo` (rate), `iip` (production). All I(1).

(a) Write R code: Johansen test, VAR vs VECM decision, fit model, forecast 6 months.

(b) `repo` Granger-causes `inf` (p=0.01). Does this prove raising repo lowers inflation?

(c) IRF of `iip` to `inf` shock: CI is very wide, crosses zero everywhere. Interpret & fix.

```r
library(vars); library(urca)
y  <- cbind(inf, repo, iip)
jo <- ca.jo(y, type="trace",
            ecdet="const", K=3)
summary(jo)
# r>0: VECM; r=0: VAR on diff(y)
if(r > 0){
  v <- vec2var(jo, r=r)
} else {
  v <- VAR(apply(y,2,diff),
           p=2, type="const")
}
plot(predict(v, n.ahead=6))
```

Solutions

(b) No. Granger = *predictive* causality only. Past repo rates help predict inflation statistically, but cannot establish structural (economic) causality. A third variable (e.g. global oil prices) may drive both. True causal inference needs SVAR with identifying restrictions.

(c) Wide CI crossing zero = IRF is **statistically indistinguishable from zero** at all horizons – insufficient evidence of any effect. Causes: (1) Only 120 obs – bootstrapped CIs will be wide; (2) Wrong lag order – re-run `VARselect()`; (3) COVID structural break 2020 – split sample; (4) Non-linear relationship – try LSTM instead.

Putting It All Together – End-to-End DL Pipeline in R

Production-Grade Workflow

1. **Data prep**: `recipes` – impute, scale, lag features
2. **Baseline**: Linear model / VAR to benchmark
3. **Classic ML**: `caret` RF/XGBoost on tabular features
4. **DL**: `keras` MLP or LSTM on sequence data
5. **Transfer**: Fine-tune pretrained CNN if applicable
6. **Evaluate**: RMSE, MAE on hold-out test set
7. **Deploy**: `vetiver` + `plumber` for REST API

```r
library(vetiver)
v <- vetiver_model(model, "lstm_v1")
vetiver_write_plumber(v, "plumber.R")
# REST API: POST /predict
```

Key R Packages Summary

Task	Package(s)
DL (high-level)	`keras`, `luz`
DL (low-level)	`torch`
Transfer learning	`keras` (built-in apps)
Classic ML	`caret`, `tidymodels`
VAR / VECM	`vars`, `urca`
ARIMAX	`forecast`
Stationarity tests	`tseries`, `urca`
Time series tidy	`fable`, `tsibble`
Model deployment	`vetiver`, `plumber`

Exam Traps

- VAR on non-stationary series $\Rightarrow$ **spurious regression**
- LSTM input: 3D `[samples, timesteps, features]`, not a matrix
- `include_top=FALSE`: must be set to keep pretrained weights
- Augment only training images, never validation/test
- Granger causality $\neq$ structural causality

Summary & Key Takeaways

Part A: R Libraries

- `keras` = high-level, TF-backed, easiest to learn
- `torch` = no Python, dynamic graph, research-grade
- `luz` = high-level wrapper for `torch`
- Choose based on: data type, sample size, Python availability

Part B: Neural Networks

- Perceptron $\rightarrow$ MLP $\rightarrow$ CNN: increasing expressiveness
- Backpropagation = chain rule for gradient computation
- ReLU default for hidden; softmax for multi-class output
- Dropout (FC layers) + BatchNorm (CNNs) prevent overfitting

Part C: Transfer Learning & ARIMA

- Reuse pretrained weights; retrain only the head initially
- Fine-tune top layers with *very small* learning rate ($\leq 10^{-5}$)
- ARIMA handles univariate forecasts; ARIMAX adds external inputs
- Augment training data; never augment validation/test

Part D: Multivariate Time Series

- Stationarity test first; VAR on stationary series
- Cointegrated I(1) series $\Rightarrow$ VECM
- Granger causality = statistical predictive causality only
- LSTM for non-linear MTS with large datasets

References & Further Reading

1. Chollet, F. & Allaire, J.J. (2018). *Deep Learning with R*. Manning Publications.
2. Shumway, R.H. & Stoffer, D.S. (2017). *Time Series Analysis and Its Applications*, 4th ed. Springer.
3. Lütkepohl, H. (2005). *New Introduction to Multiple Time Series Analysis*. Springer.
4. Keydana, S. (2023). *Deep Learning and Scientific Computing with R* `torch`. O'Reilly.
5. Hvitfeldt, E. & Silge, J. (2021). *Supervised Machine Learning for Text Analysis in R*. CRC Press.
6. Hyndman, R.J. & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice*, 3rd ed. `https://otexts.com/fpp3/`
7. R packages documentation: `vars`, `urca`, `keras`, `torch`, `forecast`, `fable`

Dept. of CSE & IT, JUIT Waknaghat
Course: Data Analytics Using R and Python (25B1BCA532)
Lecture 3 – Advanced Topics in ML, DL & Time Series